

Retrieval-Augmented Generation Note-Taking Framework

Cole Drumheller

Tuesday 28th October, 2025 - 10:15

1 Summary

This week we are looking at Retrieval-Augmented Generation or RAG. The broad idea of the RAG process is to take a user's query and add some external data to it to help the LLM give a more informed answer on whatever topic the user is asking about. One of the videos this week divided rag into 3 stages, Indexing, Retrieval, and Generation. There are also currently three main categories of RAG, naive, advanced, and modular.

Let's start with the three stages of RAG. First, the indexing stage, this is where documents are divided into meaningful chunks. This can be done in a variety of ways, one example was creating a knowledge graph in which documents are organized by topic. Then we have the retrieval stage, where like it sounds documents or data are retrieved. In this stage the input from the user is considered and used to pull in the most relevant external data. Then finally we have the generation stage, this is where the responses are created using the external data to be more accurate. (How RAG Turns AI Chatbots Into Something Practical, Bycloud).

Now let's look at the three categories of RAG starting with Naive. Naive is the earliest version of RAG and follows the three basic stages. It indexes raw data and encodes the chunks using an embedding model. Then it encodes the user query using the same embedding model and finds similar embeddings. Then it returns it all to the LLM which generates a response. This method often had issues with accurate retrieval and hallucinations during generation. Then we have Advanced RAG. This method aimed to build upon Naive RAG and fix some its issues. It adds pre and post retrieval processes. Pre-retrieval involves optimizing the user query and indexing structure. Post-retrieval does rerank of returned data to ensure we send the most useful data back to the LLM. Finally we have Modular RAG. This continues to expand upon Advanced RAG, it adds more "modules". For example, it adds a search module for similarity searches during the retrieval stage. Most of the modules focus on fine-tuning and improving the retrieval process to make sure we are getting good data. The list could go on and on for how many adaptations and things have been added to try and improve the RAG functionality.

2 Attention Grabbers

1. One thing that stood out to me in the first video is how big and fast the jump was when transformers started to be used. It only took a couple years to get chatGPT and image generation models, etc. Prior to this each 'phase' of AI was a decade or 2 long.
2. I thought the first video was interesting. It provides a very different perspective on LLMs by focusing more on the business capabilities.
3. I work for an electronics company, and I think we actually have a company LLM that I would assume works somewhat in this way. It's able to answer questions about our product line and stuff like that. Or it might be directly trained on our company data.
4. The third video talks about the 3 main stages of RAG. What stood out to me is how you can have models within models, for example having a query rewriter or trainable embedding model within the RAG steps to make it better.
5. I just think the overall concept is interesting. It's like giving a kid a textbook and asking him questions about it, he probably doesn't know the answer but you gave him the tools to find it.

3 Questions

1. Could RAG be a true replacement to training models on more and more data, instead training them on enough to get good embeddings and then using RAG to add context otherwise?
2. What are the limits of RAG, how much context can you import without severely hindering performance?
3. Do you have to train the actual RAG process to make sure it gives high quality documents/context?
4. If you had the resources to train the LLM on the documents instead of using RAG methods, would that be better?
5. Is RAG truly adding to the LLMs understanding or is it more just helping it to regurgitate external text?